Running ONNX and LiteRT models on the NPU from an Arduino App

This guide shows how an App's own Python code (`python/main.py`) can run a model on the board's Hexagon NPU, like VENTUNO Q. It covers two frameworks:

1. ONNX Runtime with the QNN execution provider
2. LiteRT (TensorFlow Lite) with the QNN delegate

Each chapter lists what to install, how to configure it, and a minimal `main.py` that starts and use an accelerated model.

How it works

What every App already has. Each App is provisioned with what it needs to reach the NPU: the NPU devices are visible inside the App's container, and the fastrpc stack, which carries the calls from the CPU to the NPU, is installed and configured at every start.

What the App installs. To run a model, the App needs a working QNN stack for the framework it uses: the framework itself plus Qualcomm's AI runtime, QAIRT (also called QNN). Each chapter below shows how to install it for its framework.

All QAIRT files must come from **the same QAIRT release**. The NPU finds its part of them through the `ADSP_LIBRARY_PATH` environment variable: if it is missing or points at the wrong files, the model can't run on the NPU. The next chapters show how to set this up for each framework.

---

1. ONNX Runtime

The `onnxruntime-qnn` package is a plugin execution provider. It ships the QAIRT release it was built with, so nothing has to be downloaded by hand.

1.1 Install

`python/requirements.txt`:

```
onnxruntime
onnxruntime-qnn
```

`onnxruntime-qnn` must match the installed `onnxruntime`. Each `onnxruntime-qnn` release declares the `onnxruntime` versions it supports, so leaving both unpinned installs a matching pair. If you pin one of them, pin the other to a version that matches it.

At the first start, `run.sh` installs them into the App's `.cache/.venv`. Later starts skip the install.

1.2 Minimal App

```
my-onnx-app/
├── app.yaml
└── python/
    ├── main.py
    ├── requirements.txt
    └── models/
        ├── mobilenet_v2.onnx
        └── mobilenet_v2.data      # external weights: keep them next to the .onnx
```

The example model is MobileNet-v2 from Qualcomm AI Hub. To fetch it, run this on the board, for example over SSH:

```
mkdir -p ~/ArduinoApps/my-onnx-app/python/models && cd ~/ArduinoApps/my-onnx-app/python/models
curl -fLO https://qaihub-public-assets.s3.us-west-2.amazonaws.com/qai-hub-
models/models/mobilenet_v2/releases/v0.64.0/mobilenet_v2-onnx-float.zip
unzip -j mobilenet_v2-onnx-float.zip "*/mobilenet_v2.onnx" "*/mobilenet_v2.data" && rm
mobilenet_v2-onnx-float.zip
```

`python/main.py`:

```python
import time

import numpy as np
import onnxruntime as ort
import onnxruntime_qnn

from arduino.app_utils import App

MODEL = "/app/python/models/mobilenet_v2.onnx"
QNN_HTP = onnxruntime_qnn.get_qnn_htp_path()  # libQnnHtp.so shipped in the wheel

ort.register_execution_provider_library("QNNExecutionProvider",
onnxruntime_qnn.get_library_path())
npu = [device for device in ort.get_ep_devices() if device.ep_name == "QNNExecutionProvider"]

options = ort.SessionOptions()
options.add_provider_for_devices(npu, {"backend_path": QNN_HTP, "htp_performance_mode":
"sustained_high_performance"})
session = ort.InferenceSession(MODEL, sess_options=options)

model_input = session.get_inputs()[0]
data = np.zeros(model_input.shape, dtype=np.float32)


def loop():
    start = time.perf_counter()
    session.run(None, {model_input.name: data})
    print(f"Inference: {(time.perf_counter() - start) * 1000:.1f} ms")
    time.sleep(1)


App.run(user_loop=loop)
```

How the code works:

- **Imports.** `onnxruntime` is the standard ONNX Runtime. `onnxruntime_qnn` is the plugin that adds the NPU to it: importing it is the first step to turn QNN on.

- **Turning QNN on.** `register_execution_provider_library()` loads the plugin into ONNX Runtime, and `get_ep_devices()` finds the NPU that the plugin exposes. `add_provider_for_devices()` tells the session to run the model on that NPU, with the options described in 1.4.
- **Loading the model.** `InferenceSession` loads the model and prepares it for the NPU. This happens once, when the App starts.
- **The loop.** `App.run()` calls `loop()` again and again. Each call runs one inference and prints how long it took. The input here is a dummy array of zeros: in a real App, pass your own data, like a camera frame, in place of `data`.

The quantized version of the model, `mobilenet_v2-onnx-w8a8.zip`, takes `uint8` input instead of `float32`: change the `dtype` of `data` to match.

1.3 Expected output

```
======== App is starting ============================
[W:onnxruntime:, qnn_execution_provider.cc:862 QnnEp] File mapped weights feature is only
available on Windows arm64 devices ...
[W:onnxruntime:Default, qnn_backend_manager.cc:330 operator()] Setting ADSP_LIBRARY_PATH =
/app/.cache/.venv/lib/python3.13/site-packages/onnxruntime_qnn
INFO - [MainThread] App:  App started
Inference: 3.0 ms
Inference: 2.6 ms
```

Both warnings are expected. The second one is the provider pointing `ADSP_LIBRARY_PATH` at the wheel.

1.4 Provider options

The `main.py` above passes these options to the QNN execution provider:

Setting	Value	Why
`backend_path` option	`onnxruntime_qnn.get_qnn_htp_path()`	**Required.** Path of the wheel's `libQnnHtp.so`. The provider also points `ADSP_LIBRARY_PATH` at its folder, so the NPU uses the QAIRT files shipped with the wheel. Without it, session creation fails with `QNN_DEVICE_ERROR_INVALID_CONFIG`.
`htp_performance_mode` option	`sustained_high_performance`	Raises the NPU clocks: about 3.5x faster than the default on MobileNet-v2.

1.5 Checking that the model runs on the NPU

Nodes the NPU can't take run on the CPU without any error, and `session.get_providers()` lists `QNNExecutionProvider` either way. To check, add this line before the `InferenceSession` is created:

```
options.add_session_config_entry("session.disable_cpu_ep_fallback", "1")
```

With it, `InferenceSession()` raises an error when any node would run on the CPU, instead of running it there.

1.6 Caching the compiled graph (optional)

Opening a session compiles the graph for the NPU, and this takes longer as the graph gets bigger. ONNX Runtime can save the compiled graph (an "EP context" model) and load it on the next start. Keep it in `/app/.cache`, which is part of the App folder on the board and survives restarts. In the `main.py` of 1.2, add `import os` and replace the `session = ort.InferenceSession(...)` line with:

```
CACHE = "/app/.cache/mobilenet_v2_ctx.onnx"

if os.path.exists(CACHE):
    session = ort.InferenceSession(CACHE, sess_options=options)
else:
    options.add_session_config_entry("ep.context_enable", "1")
    options.add_session_config_entry("ep.context_file_path", CACHE)
    options.add_session_config_entry("ep.context_embed_mode", "1")
    session = ort.InferenceSession(MODEL, sess_options=options)
```

The cached file is tied to the QAIRT release and the provider options: delete it after you change either.

---

2. LiteRT

`ai-edge-litert` has no NPU support of its own. The NPU is reached through the **QNN TFLite delegate**, which comes from the QAIRT SDK: the App must carry the delegate and the QNN libraries it needs, all from one SDK release.

2.1 Install the Python package

`python/requirements.txt`:

```
ai-edge-litert
```

2.2 Install QAIRT into the App

The App needs 5 files from the QAIRT SDK, a public zip of a few GB from Qualcomm. The script below downloads the zip, copies the 5 files into the App and deletes the zip. This example is for a VENTUNO Q, whose NPU has the Hexagon v75 architecture:

- `QAIRT_VERSION`: the QAIRT release to download.
- `HEXAGON_ARCH`: the Hexagon architecture of the NPU, `75` on the VENTUNO Q. Change it for a board with another architecture.
- `APP_DIR`: the App folder.

Run it on the board, or on a Linux or macOS machine and then copy the App folder to the board. Your home folder needs room for the zip while the script runs.

```
QAIRT_VERSION=2.50.40.260831
HEXAGON_ARCH=75
APP_DIR=~/ArduinoApps/my-litert-app

curl -fL -o ~/qairt.zip \

"https://softwarecenter.qualcomm.com/api/download/software/sdks/Qualcomm_AI_Runtime_Community/All/${QAIRT_VERSION}/v${QAIRT_VERSION}.zip"

LIB=qairt/${QAIRT_VERSION}/lib
mkdir -p "$APP_DIR/qairt/lib" "$APP_DIR/qairt/dsp"
unzip -j ~/qairt.zip -d "$APP_DIR/qairt/lib" \
  "$LIB/aarch64-oe-linux-gcc11.2/libQnnTFLiteDelegate.so" \
  "$LIB/aarch64-oe-linux-gcc11.2/libQnnHtp.so" \
  "$LIB/aarch64-oe-linux-gcc11.2/libQnnHtpPrepare.so" \
  "$LIB/aarch64-oe-linux-gcc11.2/libQnnHtpV${HEXAGON_ARCH}Stub.so"
unzip -j ~/qairt.zip -d "$APP_DIR/qairt/dsp" \
  "$LIB/hexagon-v${HEXAGON_ARCH}/unsigned/libQnnHtpV${HEXAGON_ARCH}Skel.so"
rm ~/qairt.zip
```

File	Size	Role
`lib/libQnnTFLiteDelegate.so`	1 MB	The delegate that `load_delegate()` opens.
`lib/libQnnHtp.so`	4.5 MB	QNN backend for the NPU.
`lib/libQnnHtpPrepare.so`	83 MB	Prepares the model for the NPU when it is loaded. Without it, the model runs on the CPU.
`lib/libQnnHtpV<arch>Stub.so`, like `libQnnHtpV75Stub.so`	0.5 MB	Connects the App to the NPU.
`dsp/libQnnHtpV<arch>Skel.so`, like `libQnnHtpV75Skel.so`	11-14 MB	The part of QAIRT that the NPU loads, from `ADSP_LIBRARY_PATH`.

Notes:

- Files for the wrong Hexagon architecture make the App fail at start (see Troubleshooting).
- The QAIRT libraries are covered by Qualcomm's license (`LICENSE.pdf` in the SDK zip). Check it before you share an App that contains them.

2.3 Minimal App

```
my-litert-app/
├── app.yaml
├── qairt/
│   ├── lib/
│   │   ├── libQnnTFLiteDelegate.so
│   │   ├── libQnnHtp.so
│   │   ├── libQnnHtpPrepare.so
│   │   └── libQnnHtpV75Stub.so     # V<arch>: Hexagon architecture of the NPU
│   └── dsp/
│       └── libQnnHtpV75Skel.so
└── python/
    ├── main.py
    ├── requirements.txt
    └── models/
        └── mobilenet_v2.tflite
```

The example model, from Qualcomm AI Hub:

```
mkdir -p ~/ArduinoApps/my-litert-app/python/models && cd ~/ArduinoApps/my-litert-
app/python/models
curl -fLO https://qaihub-public-assets.s3.us-west-2.amazonaws.com/qai-hub-
models/models/mobilenet_v2/releases/v0.64.0/mobilenet_v2-tflite-float.zip
unzip -j mobilenet_v2-tflite-float.zip "*/mobilenet_v2.tflite" && rm mobilenet_v2-tflite-
float.zip
```

`python/main.py`:

```
import os
import time

import numpy as np
from ai_edge_litert.interpreter import Interpreter, load_delegate

from arduino.app_utils import App

QAIRT = "/app/qairt"
MODEL = "/app/python/models/mobilenet_v2.tflite"

# Tell the NPU where to find its QAIRT files.
os.environ["ADSP_LIBRARY_PATH"] = f"{QAIRT}/dsp"

delegate = load_delegate(
    f"{QAIRT}/lib/libQnnTFLiteDelegate.so",
    {"backend_type": "htp", "htp_performance_mode": "1"},  # 1 = sustained high performance
)
interpreter = Interpreter(MODEL, experimental_delegates=[delegate])
interpreter.allocate_tensors()

model_input = interpreter.get_input_details()[0]
interpreter.set_tensor(model_input["index"], np.zeros(model_input["shape"],
dtype=model_input["dtype"]))


def loop():
    start = time.perf_counter()
    interpreter.invoke()
    print(f"Inference: {(time.perf_counter() - start) * 1000:.1f} ms")
    time.sleep(1)


App.run(user_loop=loop)
```

How the code works:

- **Imports.** `ai_edge_litert` is the standard LiteRT interpreter. Here QNN is not a Python package: it is the delegate library copied into `qairt/lib` in 2.2.
- **Turning QNN on.** `ADSP_LIBRARY_PATH` tells the NPU where to find its part of QAIRT, in `qairt/dsp`. `load_delegate()` opens the QNN delegate, with the options described in 2.5.
- **Loading the model.** Passing the delegate to `Interpreter` makes LiteRT run the model on the NPU, and `allocate_tensors()` prepares it. This happens once, when the App starts.
- **The loop.** `App.run()` calls `loop()` again and again. Each call runs one inference and prints how long it took. The input here is a dummy array of zeros, set once with `set_tensor()`: in a real App, call `set_tensor()` with your own data, like a camera frame, before each `invoke()`.

The quantized version of the model, `mobilenet_v2-tflite-w8a8.zip`, works with the same code.

2.4 Expected output

```
======== App is starting ============================
 <E> Failed to set RPC polling time. error code 512
 <E> Failed to set rpc polling time 13006
 <E> Failed to set RPC polling time
 <E> Failed to set rpc polling latency 13006
 <E> Failed to set powerConfig with error 0x32ce
INFO - [MainThread] App:  App started
Inference: 2.1 ms
Inference: 2.3 ms
```

The `<E>` lines come from `htp_performance_mode` and are harmless: the speed-up applies anyway.

2.5 Environment variables and delegate options

The `main.py` above sets this environment variable and these delegate options:

Setting	Value	Why
`ADSP_LIBRARY_PATH`	`/app/qairt/dsp`	**Required.** Without it the NPU can't find its QAIRT files (`Failed to load skel, error: 1002`) and `Interpreter()` raises `RuntimeError: Restored original execution plan after delegate application failure.` Set it before `load_delegate()` . The delegate option `"skel_library_dir": "/app/qairt/dsp"` works the same way.
`backend_type` option	`htp`	Required: runs the model on the NPU.
`htp_performance_mode` option	`1` (sustained high performance)	Raises the NPU clocks: about 3.5x faster than the default on MobileNet-v2.

2.6 Checking that the model runs on the NPU

Nodes the delegate can't take run on the CPU without any error. To check, add `"log_level": "3"` to the delegate options, start the App once and look for this line:

```
INFO: [Qnn Delegate] TfLiteQnnDelegate delegate: 71 nodes delegated out of 71 nodes with 1
partitions.
```

`0 nodes delegated` means the model is running on the CPU (for example because `libQnnHtpPrepare.so` is missing). Remove `log_level` afterwards: level 3 prints about 5000 lines at every start.

Troubleshooting

Symptom in the App log	Cause	Fix
ONNX: `QNN SetupBackend failed ... QNN_DEVICE_ERROR_INVALID_CONFIG`, CPU-like timings	The `backend_path` option is missing	Pass it, as in 1.2
ONNX: `Unable to set HTP power configurations`	`htp_performance_mode` is `burst`	Use `sustained_high_performance`
LiteRT: `Failed to load skel, error: 1002` … `error=14001`, then `RuntimeError: Restored original execution plan after delegate application failure.`	`ADSP_LIBRARY_PATH` is missing or wrong, or the files in `qairt/` come from different QAIRT releases	Point it at `/app/qairt/dsp` before `load_delegate()`, and take every file from one SDK zip
LiteRT: `Failed to load skel, error: 4000` … `error: 14001`, then the same `RuntimeError`	The files in `qairt/` are for another Hexagon architecture	Extract the ones for the board's architecture (`HEXAGON_ARCH` in 2.2)
LiteRT: `PrepareLibLoader Failed loading libQnnHtpPrepare.so`, CPU-like timings	`libQnnHtpPrepare.so` is missing	Add it to `qairt/lib`

Measured performance

MobileNet-v2 (224x224) from AI Hub v0.64.0. Median of 50 runs, measured inside the App container on the board with the options of the examples above. CPU runs: ONNX Runtime with its default threading, LiteRT with `num_threads=8`. NPU times vary from one run to the next: these are typical values.

Model	CPU	NPU, default mode	NPU, sustained high performance
ONNX float32	14.2 ms	5.2 ms	**1.5 ms**
ONNX w8a8	12.8 ms	2.0 ms	**0.8 ms**
LiteRT float32	18.3 ms	5.8 ms	**1.7 ms**
LiteRT w8a8	10.9 ms	2.1 ms	**0.6 ms**

An App that runs one inference per second, like the minimal ones above, shows 2-3 ms per call: between calls the NPU drops back to idle clocks.

Tested on

- ONNX: `onnxruntime` 1.31.0, `onnxruntime-qnn` 2.6.0 (bundles QAIRT 2.50.40)
- LiteRT: `ai-edge-litert` 2.3.0, QAIRT SDK 2.50.40.260831